Object Detection and Image Segmentation

Structured Notes

Video Fundamentals, Background

Subtraction, Optical Flow,
Bounding Box Formats, IoU, Precision/Recall, YOLO, NMS

Contents

1 Introduction to Object Detection **2**

2 Video Fundamentals **2**
2.1 Persistence of Vision . . . 2
2.2 Frames Per Second (FPS) . . . 2

3 Static and Dynamic Variations in Video **2**
3.1 Surveillance as a Key Application . . . 3

4 Object Tracking **3**

5 Object Detection Approaches **3**
5.1 Background Subtraction (Motion Detection) . . . 3
5.1.1 Step 1: Background Modeling . . . 4
5.1.2 Step 2: Frame Differencing and Thresholding . . . 4
5.1.3 Threshold Selection Methods . . . 4
5.2 Optical Flow (Motion Estimation) . . . 4

6 Challenges in Object Detection **5**

7 Deep Learning for Object Detection **5**

8 Bounding Box Representations **6**
8.1 COCO Format (Common Objects in Context) . . . 6
8.2 Pascal VOC Format (Visual Object Classes) . . . 6
8.3 YOLO Format (You Only Look Once) . . . 6

9 Evaluation Metrics for Object Detection **6**
9.1 Intersection over Union (IoU) . . . 6
9.2 Confusion Matrix for Object Detection . . . 7
9.3 Precision, Recall, and F1 Score . . . 7
9.4 Mean Average Precision (mAP) . . . 8

10 One-Stage vs. Two-Stage Detectors **8**
10.1 Two-Stage Detectors . . . 8
10.2 One-Stage Detectors . . . 8

11 YOLO (You Only Look Once) **9**

12 SSD (Single Shot Detector) **9**

13 Non-Maximum Suppression (NMS) **9**

14 Summary **10**

1 Introduction to Object Detection

Object Detection

Object detection is a computer vision technology used for detecting and locating objects within images or videos. It involves identifying what objects are present in a scene and determining their spatial locations (positions).

Object detection operates in two primary domains:

- **Image-based object detection**: Identifying and localizing objects within a single static image.
- **Video-based moving object detection**: Detecting objects that are moving across frames in a video sequence. This involves identifying dynamic variations within the video.

The core goal is to answer two questions simultaneously:

1. **What** is the object? (Classification)
2. **Where** is the object? (Localization — pixel-level position within the image or frame)

Location in Object Detection

"Location" refers to the pixel coordinates within an image or video frame where the object exists — not GPS coordinates. It is defined by the bounding region (e.g., from pixel (x_1, y_1) to pixel (x_2, y_2)) that contains the detected object.

2 Video Fundamentals

2.1 Persistence of Vision

The human eye has a persistence property: it retains an image for approximately $\frac{1}{10}$ of a second after the actual image is no longer visible. This physiological property is the foundation of video technology.

Video as Continuous Image Frames

A **video** is a continuous arrangement of image frames displayed in rapid succession. If more than 10 frames (with visual continuity) are displayed within one second, the human eye perceives them as continuous motion rather than individual discrete images.

2.2 Frames Per Second (FPS)

The minimum requirement for perceiving continuous motion is 11 or more frames per second (since the eye stores images for $\frac{1}{10}$ second). Standard broadcasting uses approximately 50 frames per second:

- Out of 50 frames: 48 are image frames and 2 are header frames (carrying metadata about the video).
- Out of the 48 image frames: 24 unique frames are repeated twice.
- Effective unique frame rate: 24 frames per second.

3 Static and Dynamic Variations in Video

When analyzing a video scene, elements are classified into:

Category	Description	Examples
Static Variation	Objects that do not change position across frames	Buildings, parked cars, poles, camera stands
Quasi-Static Variation	Background elements with minor motion	Water fountains, leaves, rain
Dynamic Variation	Objects that change position significantly across frames	People walking, vehicles moving

The objective of moving object detection is to identify the **dynamic variations** — the objects that are moving from one location to another across video frames.

3.1 Surveillance as a Key Application

Surveillance is one of the primary applications of object detection. It involves monitoring an area to:

- Identify intruders or unauthorized movement
- Track individuals moving through a scene
- Detect theft or suspicious activity
- Recognize and classify different types of objects and persons

4 Object Tracking

Object Tracking

Object tracking is the process of identifying and following the trajectory (path) of a moving object across consecutive frames in a video. While object detection identifies *where* an object is in a single frame, object tracking determines *how* that object moves over time.

To track an object, it must first be represented using a visual marker such as:

- A **bounding box** (rectangular region)
- A circle or ellipse
- A dagger (cross-shaped marker)

The tracking algorithm follows this marker across frames, recording the trajectory of the object through the video.

5 Object Detection Approaches

Object detection approaches are categorized based on whether the camera is fixed or moving:

Scenario	Camera	Objects	Approach
Fixed Camera	Stationary	Moving	Background Subtraction / Motion Detection
Moving Camera	Moving	Moving	Motion Estimation (Optical Flow)

5.1 Background Subtraction (Motion Detection)

Background subtraction is used when the camera is fixed and objects are moving through the scene. It involves two steps:

5.1.1 Step 1: Background Modeling

Construct a **background frame** X_B — an image of the scene with no objects present. This represents the static background of the scene without any moving entities.

5.1.2 Step 2: Frame Differencing and Thresholding

Compare each incoming frame X_T (the target frame at time T) with the background frame pixel by pixel:

Background Subtraction Rule:

$$\text{If } |X_T(i,j) - X_B(i,j)| > \tau \implies \text{Object pixel}$$

$$\text{If } |X_T(i,j) - X_B(i,j)| \leq \tau \implies \text{Background pixel}$$

where τ is a threshold value and (i,j) represents the pixel location.

5.1.3 Threshold Selection Methods

The threshold τ can be determined by:

- **Manual selection**: The user specifies a threshold value.
- **Average pixel value**: Compute the average pixel intensity of the difference frame.
- **Statistical method**: Use $\mu \pm k\sigma$, where μ is the mean pixel value, σ is the standard deviation, and k is a scaling factor.

Threshold Sensitivity

- **High threshold** (k large): Many parts of the actual moving object may be missed because small pixel changes fall below the threshold.
- **Low threshold** (k small): Ghost objects and noise artifacts appear — pixels that are not truly moving objects are falsely detected.

Choosing an appropriate threshold is a critical challenge in background subtraction.

5.2 Optical Flow (Motion Estimation)

Optical flow is used when both the camera and objects are moving. It estimates pixel-level motion between consecutive frames.

Optical Flow

Optical flow is the pattern of apparent motion of objects in a visual scene caused by relative motion between the observer (camera) and the scene. It computes a displacement vector (u, v) for each pixel, indicating how much and in which direction that pixel has moved between frames.

Optical flow relies on two fundamental assumptions:

Optical Flow Assumptions:

1. Brightness (Color) Constancy: A pixel's intensity value remains the same between consecutive frames, even though its position changes.

$$H(x,y) = I(x+u,\ y+v)$$

where $H(x,y)$ is the pixel value in the first frame, $I(x+u, y+v)$ is the pixel value at the displaced

position in the second frame, and (u, v) is the displacement vector.

2. Small Motion: Between consecutive frames (at 24 fps), the displacement of any object is small. This allows searching within a small rectangular neighborhood rather than the entire image.

To solve for the displacement (u, v), a **Taylor series expansion** is applied to the brightness constancy equation, enabling the estimation of pixel motion across frames.

Search Strategy in Optical Flow

For each pixel in frame H, a small rectangular search region is defined in frame I. Within this region, the algorithm searches for the pixel location where the brightness values match (i.e., color constancy holds). The size of the search region is determined by the expected maximum displacement.

6 Challenges in Object Detection

Several challenges affect the accuracy of object detection algorithms:

1. **Occlusion**: Parts of the object may be hidden behind other objects, causing incomplete detection.
2. **Ghost Objects**: Noise or thresholding errors cause non-object regions to be falsely detected as objects. These phantom detections are called ghosts.
3. **Camouflage (Similar Colors)**: When the foreground object and background have similar color/intensity values, the object may blend into the background and go undetected.
4. **Shadow Detection**: Shadows of moving objects also move in the video. Depending on the application, shadows may need to be detected as part of the object or separated from it.
5. **Vegetation and Water Changes**: Grass swaying, tree leaves moving, flowing water, and water reflections cause false dynamic detections, as these are background elements with motion.
6. **Illumination Variation**: Changes in lighting (e.g., morning vs. evening) alter pixel values across frames, causing many background pixels to appear as moving objects.
7. **Specular Reflections**: Light reflections from surfaces can cause false detections of motion.

7 Deep Learning for Object Detection

Traditional object detection methods (background subtraction, optical flow) face significant challenges under varying conditions such as rain, fog, noise, shadows, and underwater scenarios. Deep learning approaches, particularly CNNs, address these limitations by performing **end-to-end learning** for both:

- **Object classification**: Identifying *what* the object is (car, bus, truck, person, etc.)
- **Object localization**: Identifying *where* the object is (bounding box coordinates)

End-to-End Object Detection

In deep learning-based object detection, a single model is trained to simultaneously predict both the **class label** (what type of object) and the **spatial location** (bounding box coordinates) of all objects in an image or video frame. The model learns directly from labeled training data containing ground truth bounding boxes and class annotations.

The training process uses ground truth annotations where each object has:

- A class label (e.g., car, person, bus)
- A bounding box specifying its location

A **detection loss** is computed as the difference between the predicted bounding box/class and the ground truth, and the model is optimized through backpropagation over multiple iterations.

8 Bounding Box Representations

Three major bounding box representation formats are used across different datasets and algorithms:

8.1 COCO Format (Common Objects in Context)

COCO Bounding Box: (x, y, w, h)

- (x, y): Coordinates of the **top-left corner** of the bounding box
- w: Width of the bounding box
- h: Height of the bounding box

8.2 Pascal VOC Format (Visual Object Classes)

Pascal VOC Bounding Box: (x_1, y_1, x_2, y_2)

- (x_1, y_1): Coordinates of the **top-left corner**
- (x_2, y_2): Coordinates of the **bottom-right corner**

In image spatial coordinates, the y-axis increases downward and the x-axis increases to the right. Therefore:

- Top-left: (x_1, y_1)
- Top-right: (x_2, y_1)
- Bottom-left: (x_1, y_2)
- Bottom-right: (x_2, y_2)

8.3 YOLO Format (You Only Look Once)

YOLO Bounding Box: (x, y, w, h)

- (x, y): Coordinates of the **center point** of the bounding box
- w: Width of the bounding box
- h: Height of the bounding box

Format	Representation	Reference Point
COCO	(x, y, w, h)	Top-left corner
Pascal VOC	(x_1, y_1, x_2, y_2)	Top-left and bottom-right corners
YOLO	(x, y, w, h)	Center of bounding box

Bounding Box Properties

Bounding boxes can vary in:

- **Size**: Small objects vs. large objects
- **Aspect ratio**: Different length-to-width ratios
- **Position**: Different locations within the image

A single image may contain multiple bounding boxes for **multi-object detection**.

9 Evaluation Metrics for Object Detection

9.1 Intersection over Union (IoU)

Intersection over Union

Intersection over Union (IoU) is a metric that measures the overlap between the predicted bounding box $\hat{Y}$ and the ground truth bounding box Y. It quantifies how well the predicted detection matches the actual object location.

IoU Formula:

$$\text{IoU} = \frac{Y \cap \hat{Y}}{Y \cup \hat{Y}} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

where:

- $Y \cap \hat{Y}$: The intersection area — pixels that belong to **both** the ground truth and predicted bounding boxes.
- $Y \cup \hat{Y}$: The union area — all pixels that belong to **either** the ground truth or predicted bounding box (or both).

Range: $\text{IoU} \in [0, 1]$

- $\text{IoU} = 1$: Perfect match (predicted box exactly overlaps ground truth)
- $\text{IoU} = 0$: No overlap between predicted and ground truth boxes
- Higher IoU indicates better detection accuracy

IoU vs. MSE

IoU and Mean Squared Error (MSE) are fundamentally different metrics. IoU measures spatial overlap between bounding box regions, while MSE measures the average squared difference between predicted and actual numerical values. They are not conceptually interchangeable.

9.2 Confusion Matrix for Object Detection

In object detection, each pixel (or detected region) is classified into one of four categories based on its actual class and predicted class:

	Predicted: Object (Positive)	**Predicted: Background (Negative)**
Actual: Object (Positive)	True Positive (TP)	False Negative (FN)
Actual: Background (Negative)	False Positive (FP)	True Negative (TN)

- **True Positive (TP)**: Pixel is actually an object and correctly identified as an object.
- **True Negative (TN)**: Pixel is actually background and correctly identified as background.
- **False Positive (FP)**: Pixel is actually background but *falsely* identified as an object.
- **False Negative (FN)**: Pixel is actually an object but *falsely* identified as background.

9.3 Precision, Recall, and F1 Score

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision measures: out of all instances *predicted* as objects, how many are *truly* objects. It focuses on the accuracy of positive predictions.

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall measures: out of all *actual* objects, how many were correctly identified as objects. It focuses on the completeness of detection.

F1 Score (F-Measure):

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 score is the harmonic mean of precision and recall, providing a single balanced metric.

9.4 Mean Average Precision (mAP)

Mean Average Precision

Mean Average Precision (mAP) evaluates object detection performance across multiple classes. For each class, a precision-recall curve is plotted and the **Area Under the Curve (AUC)** is computed. The mAP is the **mean** of these AUC values across all object classes.

The computation steps:

1. For each object class (e.g., car, person, bus), compute precision and recall at various threshold levels.
2. Plot the precision-recall curve for each class.
3. Calculate the area under each precision-recall curve (this is the Average Precision for that class).
4. Compute the mean of all per-class Average Precision values to obtain mAP.

10 One-Stage vs. Two-Stage Detectors

Deep learning-based bounding box detectors are divided into two categories:

10.1 Two-Stage Detectors

1. **Stage 1 — Proposal Generation**: CNN features are extracted from the input image. A proposal generator identifies possible regions where objects may be present (region proposals).
2. **Stage 2 — Classification and Detection**: Each proposed region is processed through a regression head (for bounding box refinement) and a classification head (for object class prediction).

Examples: R-CNN, Fast R-CNN, Faster R-CNN.

Limitation: The two-stage process takes more computation time, making real-time detection difficult.

10.2 One-Stage Detectors

One-stage detectors skip the proposal generation step entirely. They extract CNN features and directly predict both the bounding box coordinates and object class in a single forward pass.

Examples: YOLO (You Only Look Once), SSD (Single Shot Detector).

Advantage: Significantly faster, enabling real-time object detection.

Aspect	Two-Stage Detector	One-Stage Detector
Pipeline	Proposal generation → Detection	Direct detection
Speed	Slower (two passes)	Faster (single pass)
Real-time	Difficult	Suitable
Examples	R-CNN, Faster R-CNN	YOLO, SSD

11 YOLO (You Only Look Once)

YOLO

YOLO (You Only Look Once) is a one-stage object detection algorithm proposed by Redmon et al. in 2015. It performs object detection as a single regression problem — predicting bounding box coordinates and class probabilities directly from the full image in a single forward pass through the network.

Key characteristics of YOLO:

- **Single forward pass**: The entire image is processed once through the network, which simultaneously predicts bounding boxes and class labels.
- **Real-time capable**: Due to its single-pass architecture, YOLO achieves detection speeds suitable for real-time applications (e.g., traffic monitoring).
- **Fixed-size grid**: The input image is divided into a fixed-size grid, and each grid cell is responsible for predicting objects whose center falls within that cell.
- **Dense predictions**: YOLO makes predictions over multiple feature maps, allowing detection of objects at different scales.

YOLO was developed as an alternative to two-stage detectors like R-CNN, which were too slow for real-time applications due to the separate proposal generation step.

YOLO Versions

Multiple versions of YOLO have been developed: YOLOv1 (2015), YOLOv2, YOLOv3, YOLOv4, YOLOv5, YOLOv6, YOLOv7, and YOLOv8. Each version introduces architectural improvements for better accuracy and speed. The original version (YOLOv1/YOLO v0) uses a fixed-size grid for detection.

12 SSD (Single Shot Detector)

SSD

SSD (Single Shot Detector) is a one-stage object detection algorithm that makes dense predictions over multiple feature maps at different scales. Like YOLO, it predicts bounding boxes and class labels in a single forward pass.

SSD and YOLO are the two most widely used single-stage detection architectures. Both aim to achieve real-time detection by eliminating the proposal generation stage.

13 Non-Maximum Suppression (NMS)

Non-Maximum Suppression

Non-Maximum Suppression (NMS) is a post-processing technique used to eliminate redundant overlapping bounding box detections. When a detector produces multiple overlapping boxes for the same object, NMS retains only the most confident detection and suppresses the rest.

The NMS algorithm works as follows:

1. The detector produces multiple bounding boxes with associated confidence scores for a given image.
2. Select the bounding box with the **highest IoU** with the ground truth (or highest confidence score).
3. Compute the IoU between this selected box and all remaining boxes.

4. For each remaining box:
   - If IoU $>$ threshold (e.g., 0.3 or 0.5): The boxes overlap significantly, indicating they likely correspond to the **same object**. **Suppress** (remove) the lower-confidence box.
   - If IoU $<$ threshold: The boxes do not overlap much, indicating they correspond to **different objects**. **Keep** both boxes.
5. Repeat the process with the remaining boxes until all detections have been processed.

NMS Threshold

The IoU threshold for NMS (typically 0.3 to 0.5) determines how aggressively overlapping detections are suppressed. A lower threshold suppresses more boxes (stricter), while a higher threshold retains more overlapping detections (more lenient).

14 Summary

- **Object detection** identifies and localizes objects in images or video frames, answering both "what" and "where."
- A **video** is a continuous arrangement of image frames; the human eye requires 11+ frames/second to perceive continuous motion due to persistence of vision ($\frac{1}{10}$ second retention).
- **Background subtraction** detects moving objects by comparing each frame against a static background model using pixel-wise differencing and thresholding: $|X_T - X_B| > \tau$.
- **Optical flow** estimates pixel motion between frames using brightness constancy ($H(x, y) = I(x + u, y + v)$) and small motion assumptions.
- Key challenges include occlusion, ghost objects, camouflage, shadows, illumination variation, and vegetation/water motion.
- Three bounding box formats exist: **COCO** (x, y, w, h) from top-left, **Pascal VOC** (x_1, y_1, x_2, y_2) corners, and **YOLO** (x, y, w, h) from center.
- **IoU** $= \frac{Y \cap \hat{Y}}{Y \cup \hat{Y}}$ measures detection accuracy; ranges from 0 (no overlap) to 1 (perfect match).
- **Precision** $= \frac{TP}{TP+FP}$, **Recall** $= \frac{TP}{TP+FN}$, and **F1** $= \frac{2 \cdot P \cdot R}{P+R}$ evaluate detection quality.
- **mAP** averages the area under precision-recall curves across all object classes.
- **Two-stage detectors** (R-CNN, Faster R-CNN) use proposal generation then classification; **one-stage detectors** (YOLO, SSD) predict directly in a single pass.
- **YOLO** (Redmon et al., 2015) performs single-pass detection predicting bounding boxes and class labels simultaneously, enabling real-time detection.
- **NMS** removes redundant overlapping detections by suppressing boxes with IoU above a threshold (0.3–0.5) relative to the highest-confidence detection.